Experiment 3

Aim: To create an Employee–Department–Project database schema, insert at least 30 employees across 5 departments and 8 projects, and execute SQL queries demonstrating selection, projection, aggregation, GROUP BY, HAVING, CASE expressions, and ORDER BY.

Source Code:

```
-- ========================================================
-- 1. CREATE DEPARTMENT TABLE
-- ========================================================
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50),
    location VARCHAR(50)
);
-- ========================================================
-- 2. CREATE EMPLOYEE TABLE
-- ========================================================
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary DECIMAL(10,2),
    age INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
-- ========================================================
-- 3. CREATE PROJECT TABLE
-- ========================================================
CREATE TABLE Project (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(100),
    budget DECIMAL(12,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
```

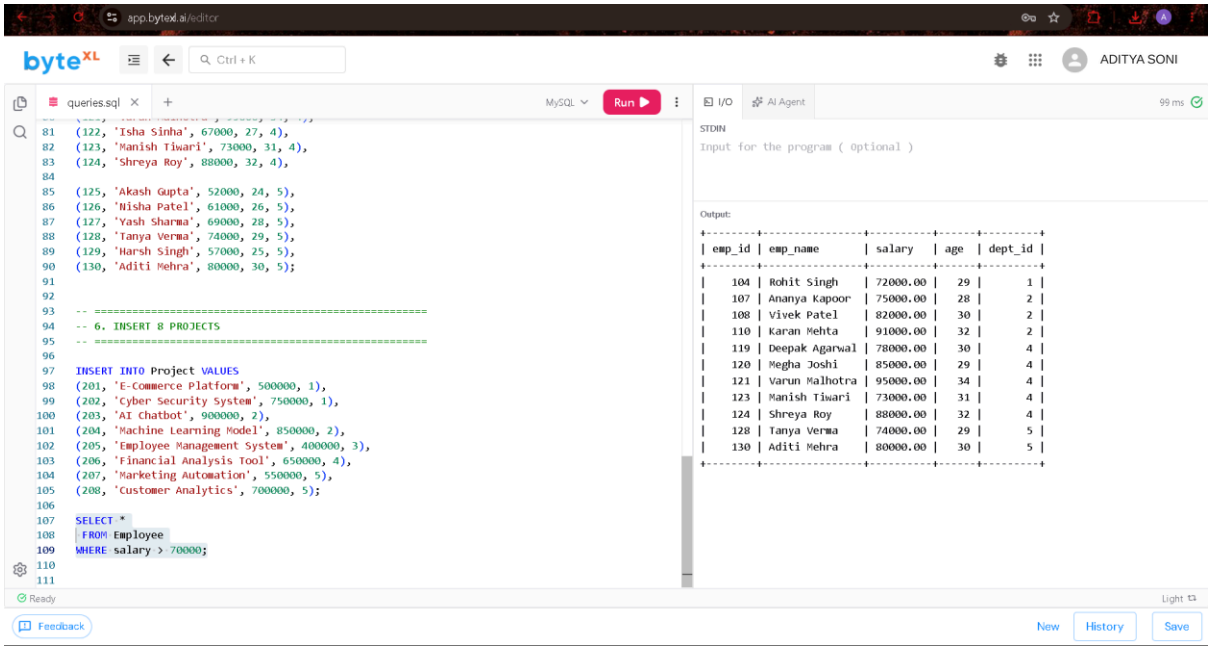
```
);
-- ========================================================
-- 4. INSERT 5 DEPARTMENTS
-- ========================================================
INSERT INTO Department VALUES
(1, 'CSE', 'Delhi'),
(2, 'AIML', 'Noida'),
(3, 'HR', 'Lucknow'),
(4, 'Finance', 'Mumbai'),
(5, 'Marketing', 'Bangalore');
-- ========================================================
-- 5. INSERT 30 EMPLOYEES
-- ========================================================
INSERT INTO Employee VALUES
(101, 'Rahul Sharma', 55000, 24, 1),
(102, 'Aman Verma', 62000, 26, 1),
(103, 'Neha Gupta', 58000, 25, 1),
(104, 'Rohit Singh', 72000, 29, 1),
(105, 'Priya Mishra', 68000, 27, 1),
(106, 'Arjun Yadav', 49000, 23, 1),
(107, 'Ananya Kapoor', 75000, 28, 2),
(108, 'Vivek Patel', 82000, 30, 2),
(109, 'Sneha Jain', 65000, 26, 2),
(110, 'Karan Mehta', 91000, 32, 2),
(111, 'Simran Kaur', 70000, 27, 2),
(112, 'Aditya Rao', 60000, 25, 2),
(113, 'Pooja Singh', 45000, 24, 3),
(114, 'Nitin Kumar', 52000, 28, 3),
(115, 'Riya Sharma', 58000, 29, 3),
(116, 'Mohit Verma', 63000, 31, 3),
(117, 'Kavita Gupta', 55000, 26, 3),
(118, 'Sahil Khan', 48000, 25, 3),
(119, 'Deepak Agarwal', 78000, 30, 4),
(120, 'Megha Joshi', 85000, 29, 4),
```

```
(121, 'Varun Malhotra', 95000, 34, 4),
(122, 'Isha Sinha', 67000, 27, 4),
(123, 'Manish Tiwari', 73000, 31, 4),
(124, 'Shreya Roy', 88000, 32, 4),
(125, 'Akash Gupta', 52000, 24, 5),
(126, 'Nisha Patel', 61000, 26, 5),
(127, 'Yash Sharma', 69000, 28, 5),
(128, 'Tanya Verma', 74000, 29, 5),
(129, 'Harsh Singh', 57000, 25, 5),
(130, 'Aditi Mehra', 80000, 30, 5);
-- ========================================================
-- 6. INSERT 8 PROJECTS
-- ========================================================
INSERT INTO Project VALUES
(201, 'E-Commerce Platform', 500000, 1),
(202, 'Cyber Security System', 750000, 1),
(203, 'AI Chatbot', 900000, 2),
(204, 'Machine Learning Model', 850000, 2),
(205, 'Employee Management System', 400000, 3),
(206, 'Financial Analysis Tool', 650000, 4),
(207, 'Marketing Automation', 550000, 5),
(208, 'Customer Analytics', 700000, 5);
```

OUTPUT:

Selection: Display employees with salary greater than 70000.

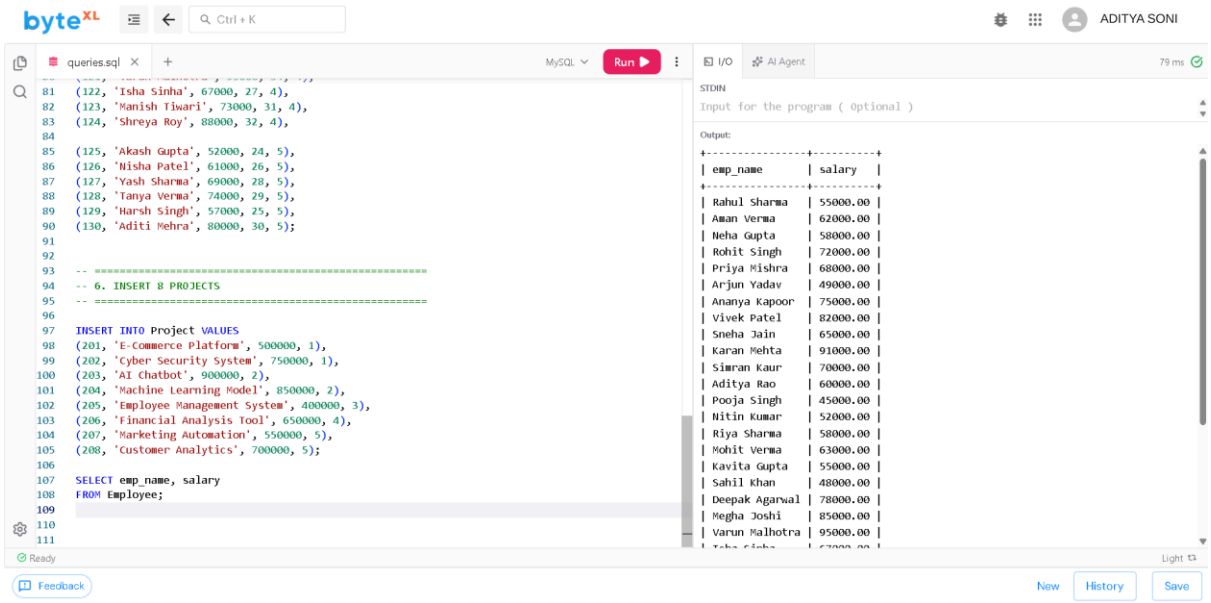
```
SELECT *
 FROM Employee
WHERE salary > 70000;
```



Projection: Display employee names and salaries.

SELECT emp_name, salary

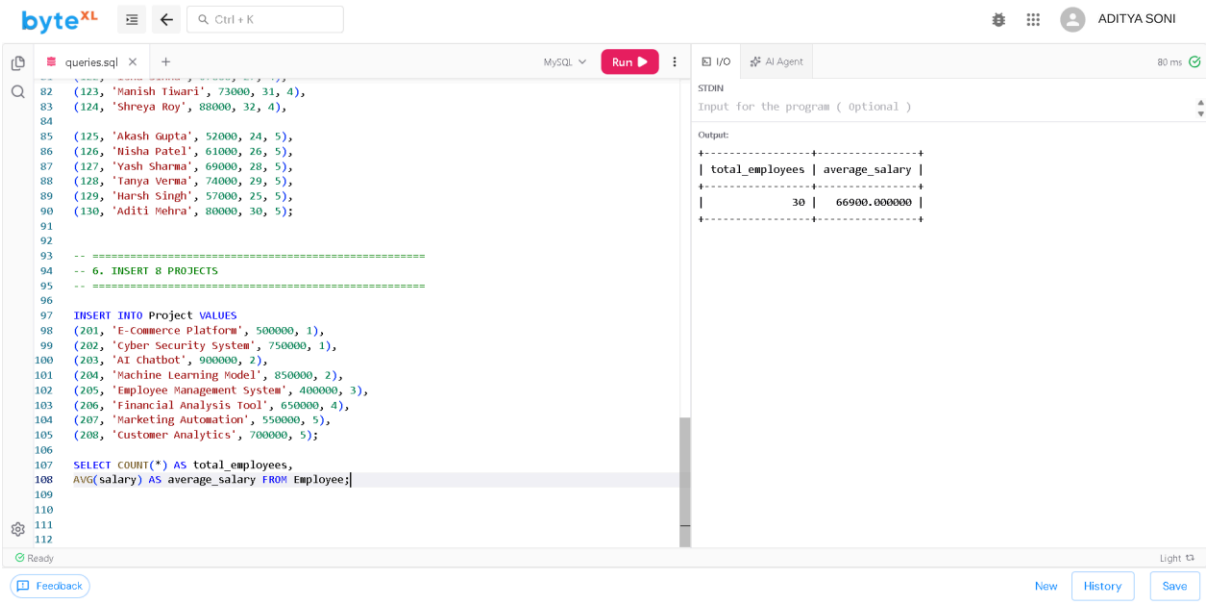
FROM Employee;



Aggregation: Count employees and calculate average salary

SELECT COUNT(*) AS total_employees,

AVG(salary) AS average_salary FROM Employee;

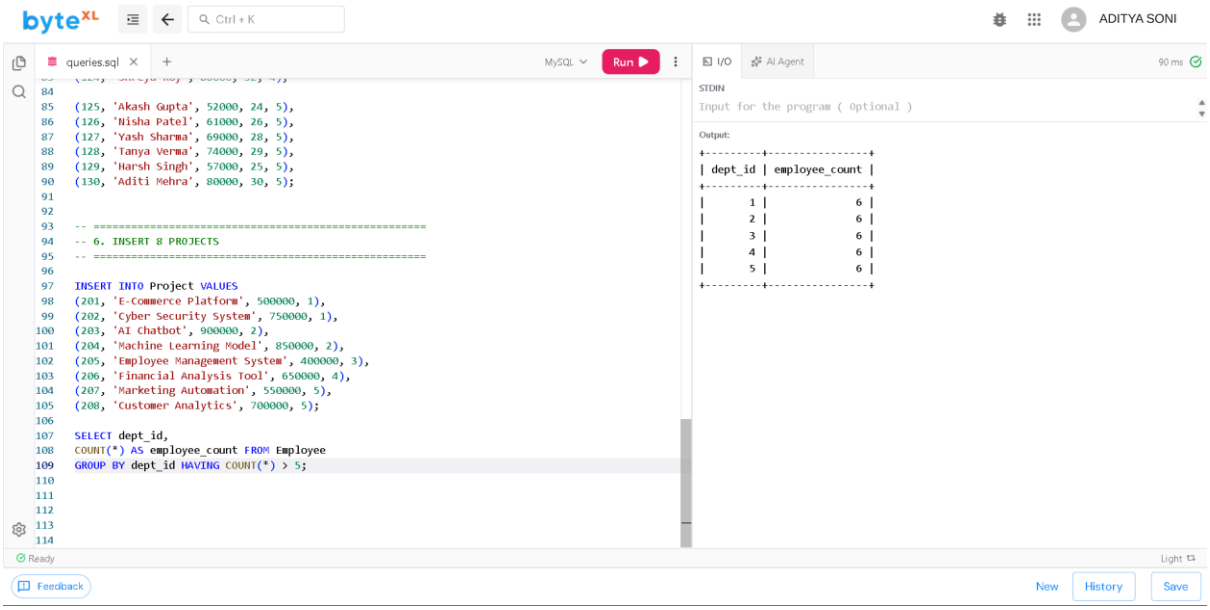


Having: departments having more than 5 employees

SELECT dept_id,

COUNT(*) AS employee_count FROM Employee

GROUP BY dept_id HAVING COUNT(*) > 5;



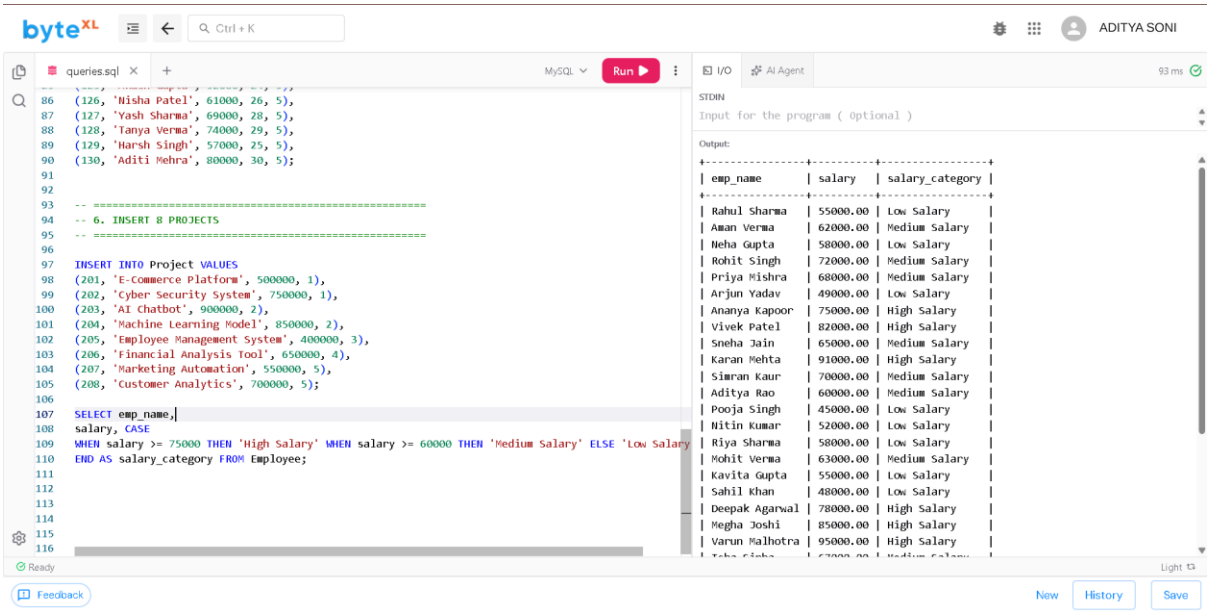
Case expression: Classify employees according to salary

SELECT emp_name,

salary, CASE

WHEN salary >= 75000 THEN 'High Salary' WHEN salary >= 60000 THEN 'Medium Salary' ELSE 'Low Salary'

END AS salary_category FROM Employee;

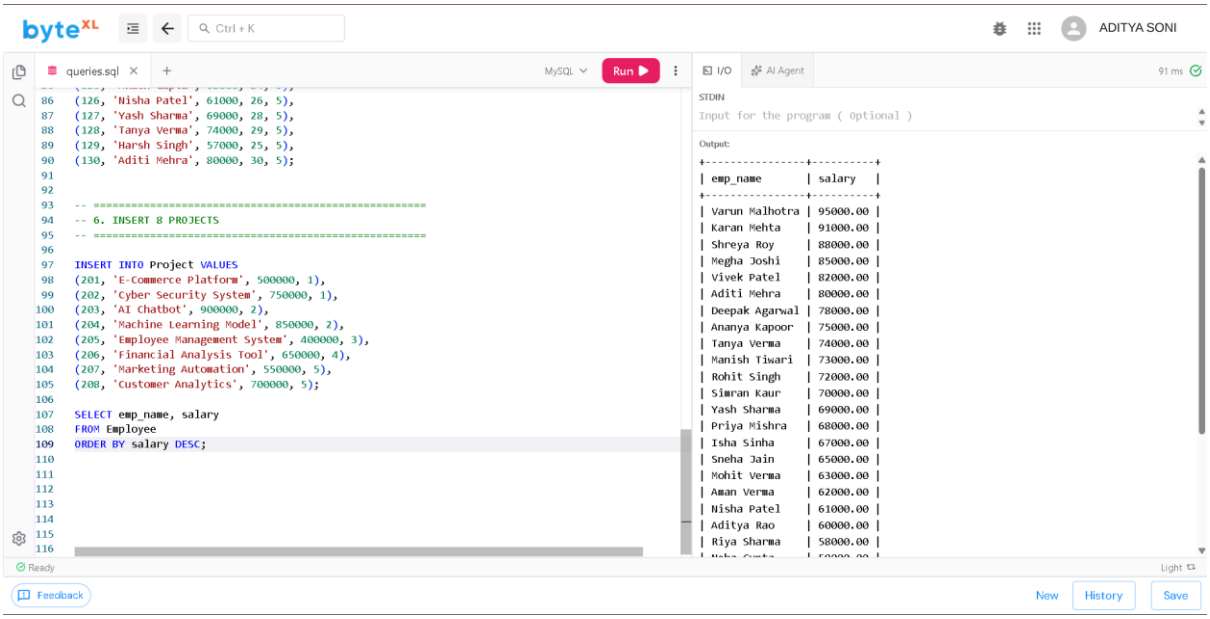


Order By: Display employee in descending order of salary

SELECT emp_name, salary

FROM Employee

ORDER BY salary DESC;



Experiment 4

Aim: Using the Employee schema, write queries with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare execution plans using EXPLAIN.

Source Code:

```
-- ------------------------------------------------------------
-- 1. CREATE TABLES
-- ------------------------------------------------------------
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50),
    location VARCHAR(50)
);
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary DECIMAL(10,2),
    job_title VARCHAR(50),
    dept_id INT,
    manager_id INT NULL,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
    FOREIGN KEY (manager_id) REFERENCES Employee(emp_id)
);
CREATE TABLE Project (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(100),
    dept_id INT,
    budget DECIMAL(12,2),
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
CREATE TABLE Employee_Project (
    emp_id INT,
    project_id INT,
    hours_worked INT,
```

```
    PRIMARY KEY (emp_id, project_id),
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),
    FOREIGN KEY (project_id) REFERENCES Project(project_id)
);
-- ============================================================
-- 2. INSERT DATA
-- ============================================================
INSERT INTO Department VALUES
(1, 'IT', 'Delhi'),
(2, 'HR', 'Mumbai'),
(3, 'Finance', 'Bangalore'),
(4, 'Marketing', 'Pune'),
(5, 'AI/ML', 'Hyderabad');
INSERT INTO Employee VALUES
(101, 'Rahul', 75000, 'Developer', 1, NULL),
(102, 'Aman', 65000, 'Developer', 1, 101),
(103, 'Priya', 90000, 'Manager', 2, NULL),
(104, 'Neha', 60000, 'HR Executive', 2, 103),
(105, 'Rohit', 85000, 'Analyst', 3, NULL),
(106, 'Sneha', 55000, 'Accountant', 3, 105),
(107, 'Karan', 70000, 'Marketing Executive', 4, NULL),
(108, 'Pooja', 50000, 'Marketing Executive', 4, 107),
(109, 'Arjun', 95000, 'AI Engineer', 5, NULL),
(110, 'Simran', 80000, 'ML Engineer', 5, 109),
(111, 'Vikas', 62000, 'Developer', 1, 101),
(112, 'Anjali', 58000, 'Developer', 1, 101),
(113, 'Mohit', 72000, 'HR Executive', 2, 103),
(114, 'Riya', 68000, 'Analyst', 3, 105),
(115, 'Varun', 78000, 'AI Engineer', 5, 109),
(116, 'Tanya', 52000, 'Marketing Executive', 4, 107),
(117, 'Aditya', 88000, 'Developer', 1, 101),
(118, 'Nisha', 61000, 'Accountant', 3, 105),
(119, 'Yash', 73000, 'ML Engineer', 5, 109),
(120, 'Kriti', 57000, 'HR Executive', 2, 103),
```

(121, 'Dev', 67000, 'Developer', 1, 101),

(122, 'Isha', 54000, 'Marketing Executive', 4, 107),

(123, 'Manish', 91000, 'AI Engineer', 5, 109),

(124, 'Ayesha', 63000, 'Analyst', 3, 105),

(125, 'Kabir', 76000, 'Developer', 1, 101),

(126, 'Meera', 59000, 'HR Executive', 2, 103),

(127, 'Sahil', 69000, 'Accountant', 3, 105),

(128, 'Diya', 81000, 'ML Engineer', 5, 109),

(129, 'Harsh', 71000, 'Marketing Executive', 4, 107),

(130, 'Muskan', 56000, 'HR Executive', 2, 103);

INSERT INTO Project VALUES

(201, 'Website Development', 1, 500000),

(202, 'Mobile App', 1, 700000),

(203, 'Recruitment System', 2, 300000),

(204, 'Financial Analysis', 3, 600000),

(205, 'Advertisement Campaign', 4, 400000),

(206, 'AI Chatbot', 5, 900000),

(207, 'Machine Learning Model', 5, 1000000),

(208, 'Data Analytics', 3, 800000);

INSERT INTO Employee_Project VALUES

(101,201,120),

(102,201,100),

(111,202,90),

(112,202,80),

(117,201,110),

(125,202,100),

(103,203,70),

(104,203,90),

(113,203,80),

(120,203,60),

(105,204,100),

(106,204,90),

(114,208,110),

(118,208,80),

(124,204,70),

(127,208,90),

(107,205,100),

(108,205,80),

(116,205,70),

(122,205,60),

(129,205,90),

(109,206,120),

(110,206,100),

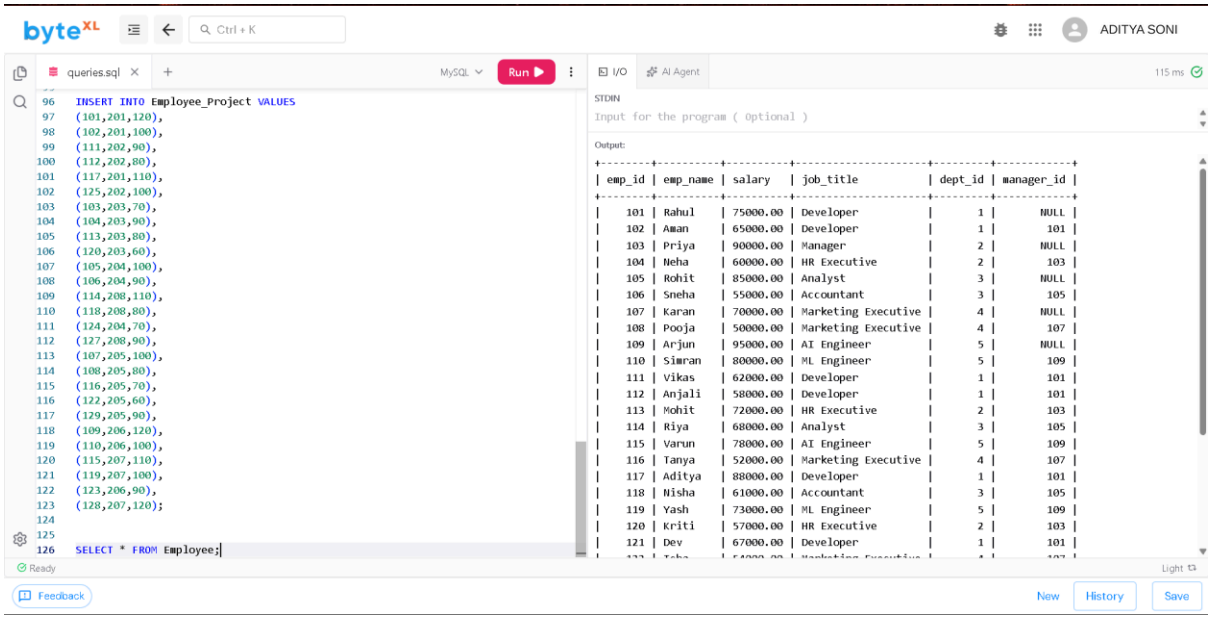
(115,207,110),

(119,207,100),

(123,206,90),

(128,207,120);

SELECT * FROM Employee;



OUTPUT:

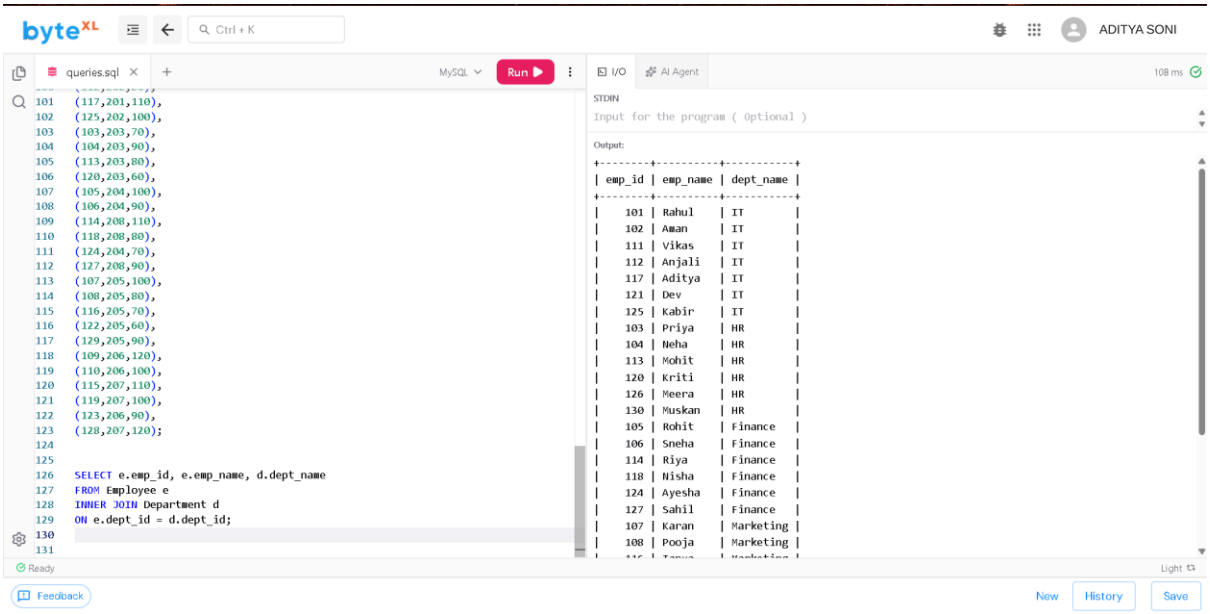
INNER JOIN:

SELECT e.emp_id, e.emp_name, d.dept_name

FROM Employee e

INNER JOIN Department d

ON e.dept_id = d.dept_id;



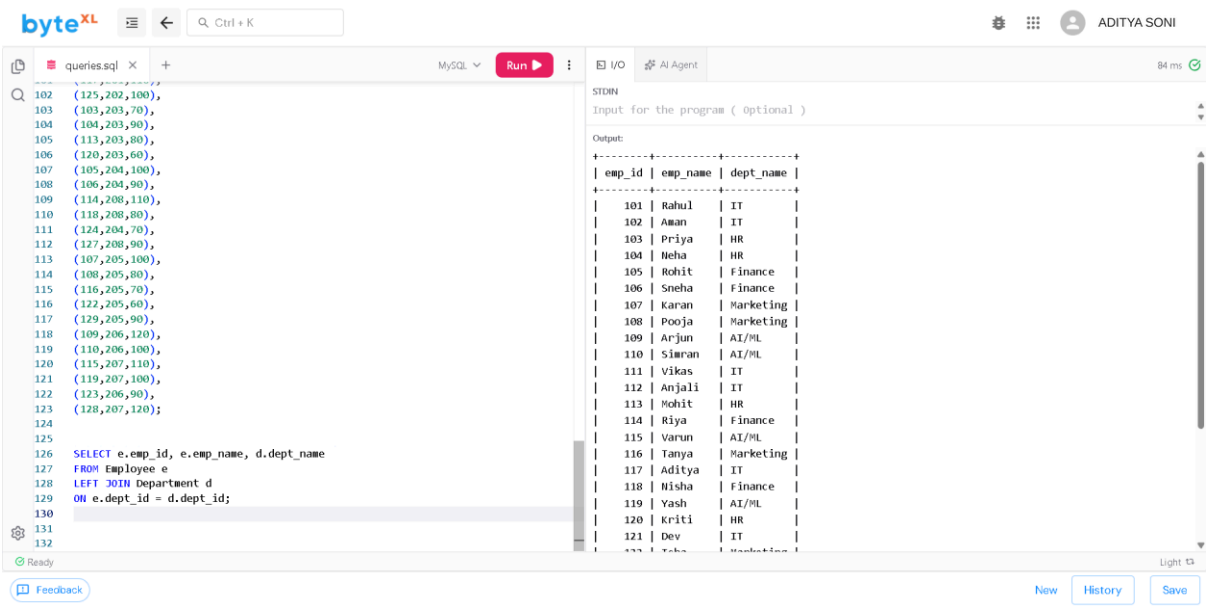
LEFT JOIN:

SELECT e.emp_id, e.emp_name, d.dept_name

FROM Employee e

LEFT JOIN Department d

ON e.dept_id = d.dept_id;



SELF JOIN:

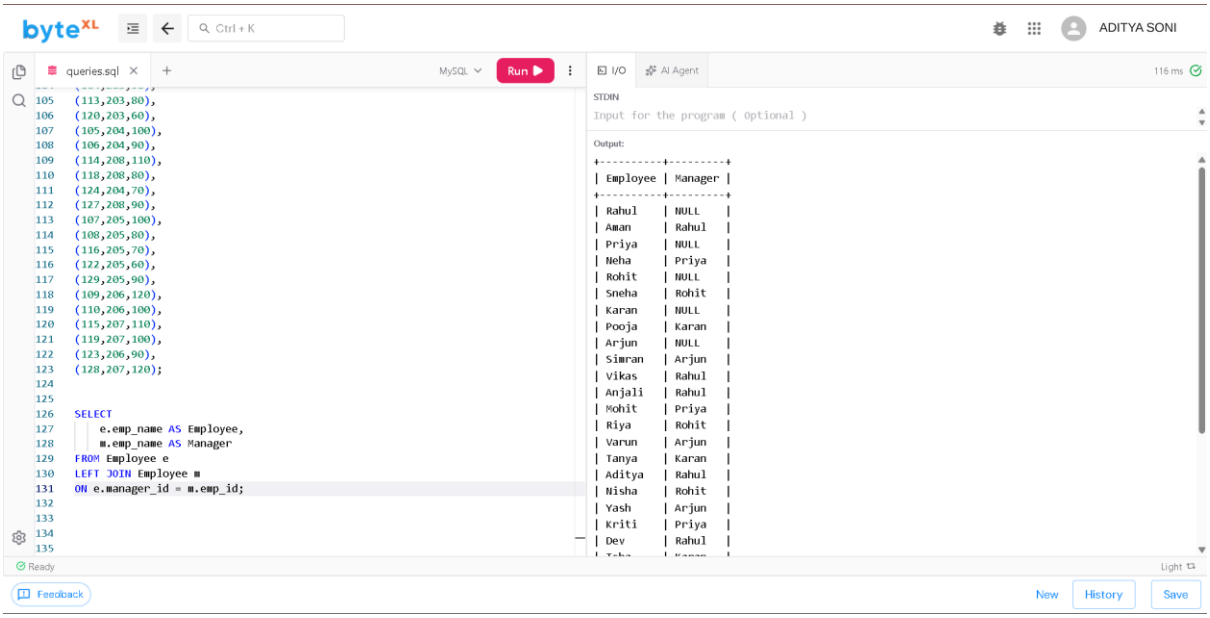
SELECT

e.emp_name AS Employee, m.emp_name AS Manager

FROM Employee e

LEFT JOIN Employee m

ON e.manager_id = m.emp_id;



THREE-WAY JOIN:

SELECT e.emp_name, d.dept_name, p.project_name, ep.hours_worked

FROM Employee e

INNER JOIN Department d

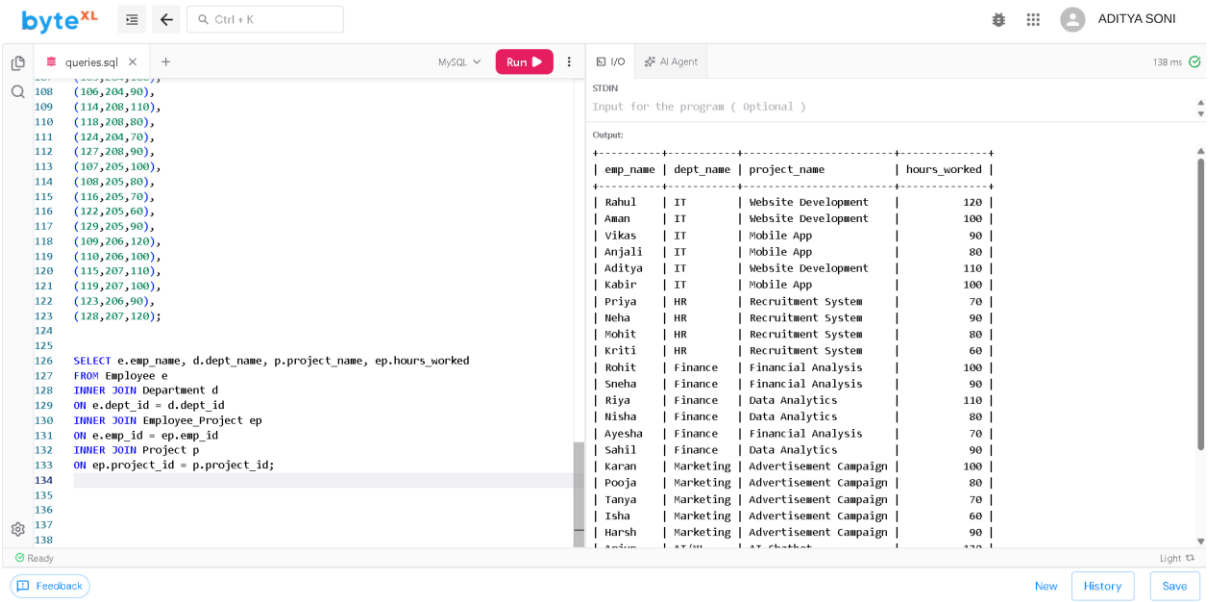
ON e.dept_id = d.dept_id

INNER JOIN Employee_Project ep

ON e.emp_id = ep.emp_id

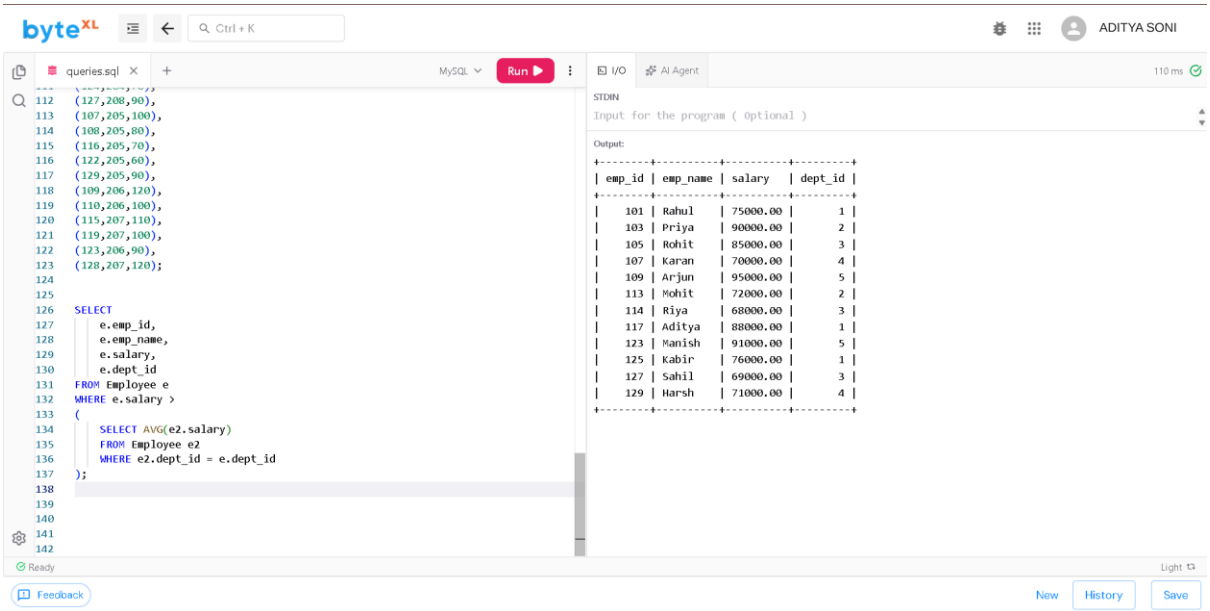
INNER JOIN Project p

ON ep.project_id = p.project_id;



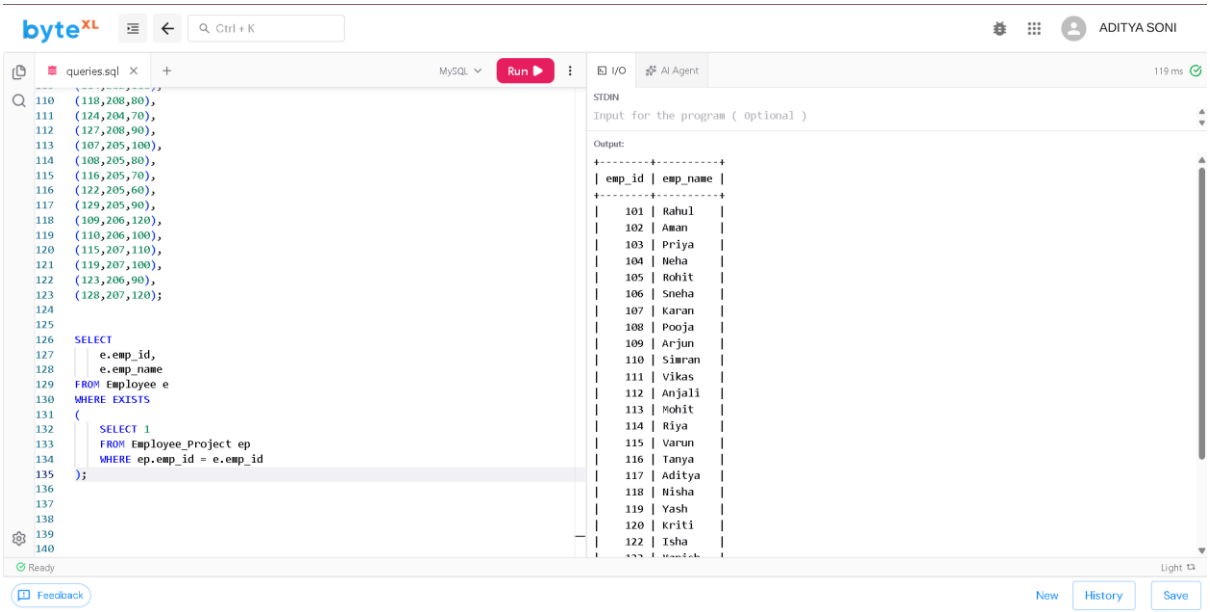
CORRELATED SUBQUERY:

```
SELECT e.emp_id, e.emp_name, e.salary, e.dept_id
FROM Employee e
WHERE e.salary > ( SELECT AVG(e2.salary)
 FROM Employee e2
WHERE e2.dept_id = e.dept_id );
```



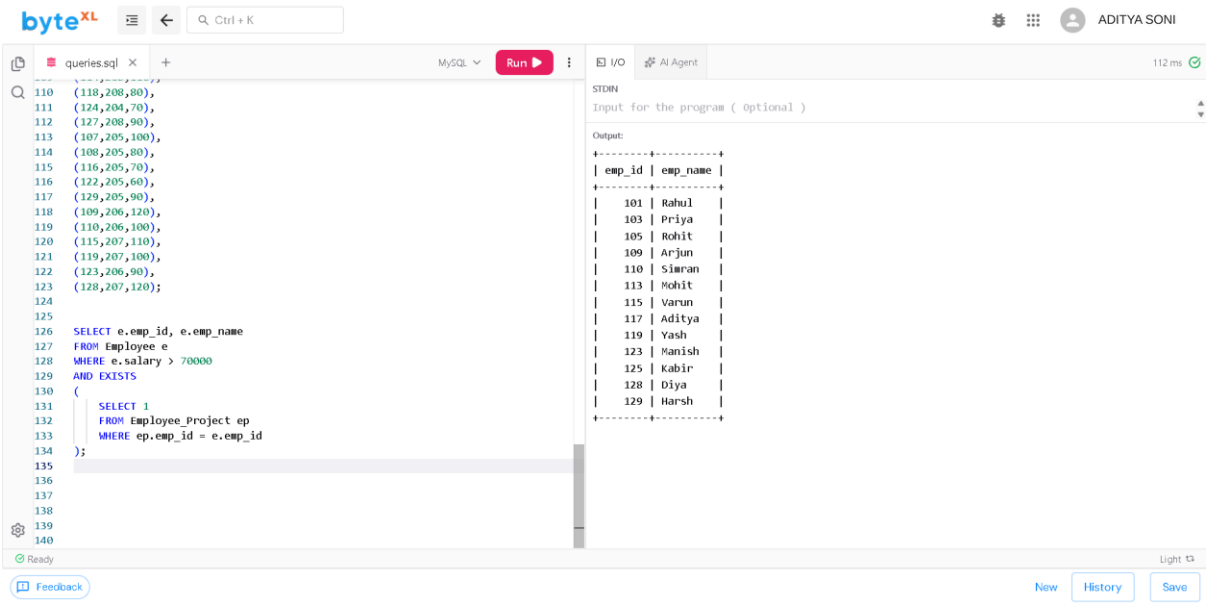
EXISTS:

```
SELECT e.emp_id, e.emp_name
FROM Employee e
WHERE EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```



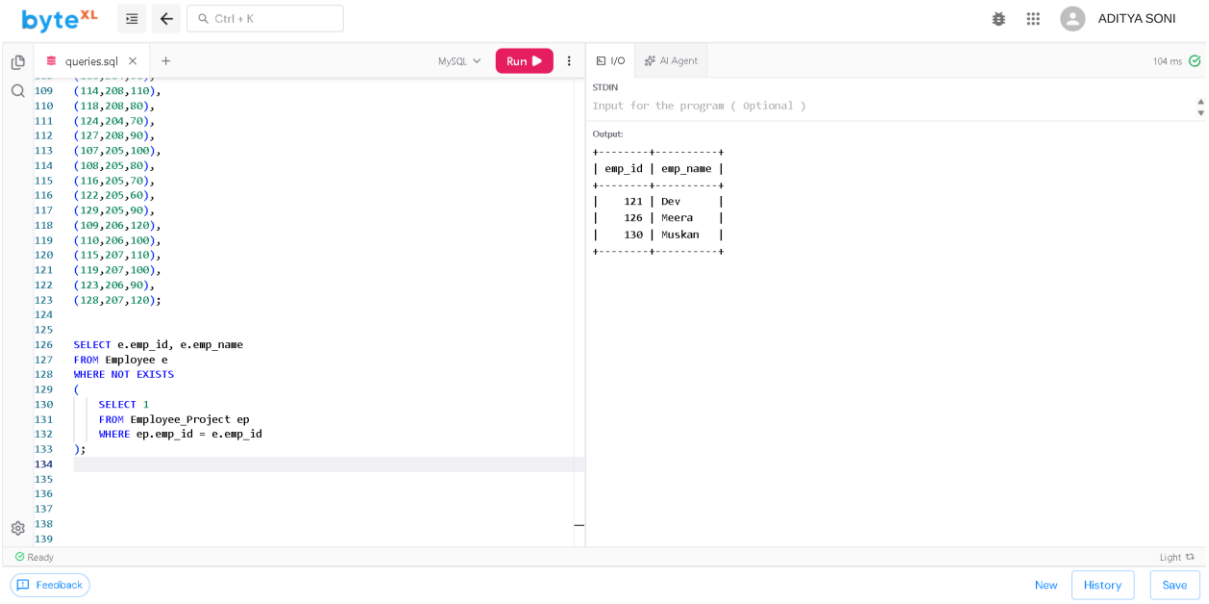
SIMULATED INTERSECT:

```
SELECT e.emp_id, e.emp_name
FROM Employee e
WHERE e.salary > 70000
AND EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```



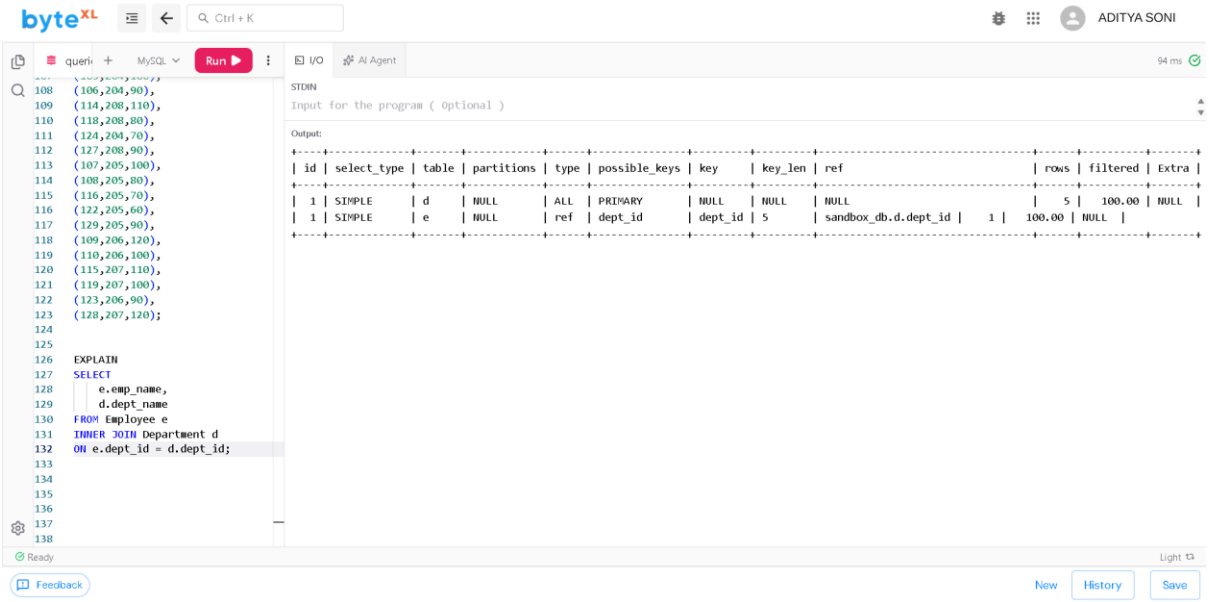
SIMULATED EXCEPT:

```
SELECT e.emp_id, e.emp_name
FROM Employee e
WHERE NOT EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```



EXPLAIN:

```
EXPLAIN
SELECT e.emp_name, d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;
```



Experiment 5

Aim: To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

Source Code:

```
-- ============================================
-- 1. CREATE DEPARTMENT TABLE
-- ============================================
CREATE TABLE department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL
);
-- Insert departments
INSERT INTO department VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');
-- ============================================
-- 2. CREATE EMPLOYEE TABLE
-- ============================================
CREATE TABLE employee (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(100) NOT NULL,
    department_id INT,
    manager_id INT NULL,
    salary DECIMAL(10,2),
    FOREIGN KEY (department_id)
        REFERENCES department(department_id),
    FOREIGN KEY (manager_id)
        REFERENCES employee(employee_id)
);
-- Insert employees
INSERT INTO employee VALUES
(101, 'Amit', 1, NULL, 90000),
(102, 'Neha', 1, 101, 65000),
```

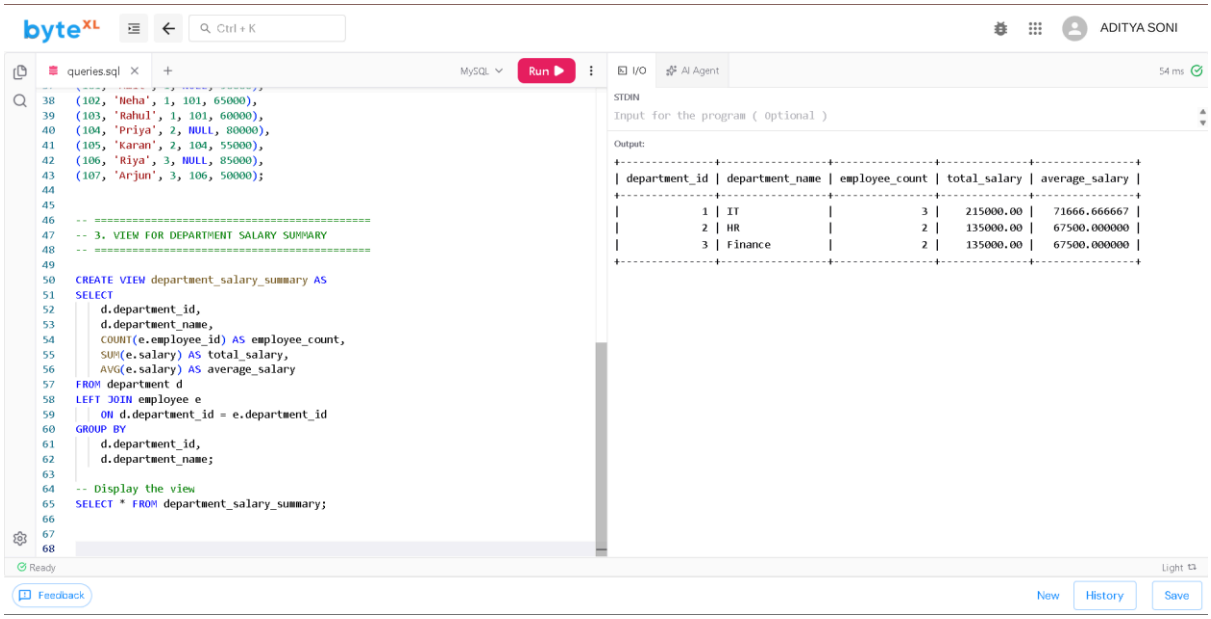
```
(103, 'Rahul', 1, 101, 60000),
(104, 'Priya', 2, NULL, 80000),
(105, 'Karan', 2, 104, 55000),
(106, 'Riya', 3, NULL, 85000),
(107, 'Arjun', 3, 106, 50000);
-- ============================================
-- 3. VIEW FOR DEPARTMENT SALARY SUMMARY
-- ============================================
CREATE VIEW department_salary_summary AS
SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS employee_count,
    SUM(e.salary) AS total_salary,
    AVG(e.salary) AS average_salary
FROM department d
LEFT JOIN employee e
    ON d.department_id = e.department_id
GROUP BY
    d.department_id,
    d.department_name;
-- Display the view
SELECT * FROM department_salary_summary;
-- ============================================
-- 4. VIEW FOR EMPLOYEE HIERARCHY
-- ============================================
CREATE VIEW employee_hierarchy AS
SELECT
    e.employee_id,
    e.employee_name,
    e.department_id,
    e.manager_id,
    m.employee_name AS manager_name,
    e.salary
```

```
FROM employee e
LEFT JOIN employee m
    ON e.manager_id = m.employee_id;
-- Display the view
SELECT * FROM employee_hierarchy;
-- ============================================
-- 5. TEST VIEW UPDATABILITY
-- ============================================
-- Simple single-table view
CREATE VIEW employee_basic AS
SELECT
    employee_id,
    employee_name,
    salary
FROM employee;
-- Update salary through the view
UPDATE employee_basic
SET salary = salary + 5000
WHERE employee_id = 102;
-- Check updated record
SELECT *
FROM employee_basic
WHERE employee_id = 102;
-- Check original table also
SELECT *
FROM employee
WHERE employee_id = 102;
-- ============================================
-- 6. RECURSIVE CTE FOR REPORTING CHAINS
-- ============================================
WITH RECURSIVE reporting_chain AS (
    -- Anchor member:
    -- Employees who have no manager
    SELECT
```

```
        employee_id,
        employee_name,
        manager_id,
        0 AS level,
        CAST(employee_name AS CHAR(1000)) AS reporting_path
    FROM employee
    WHERE manager_id IS NULL
    UNION ALL
    -- Recursive member:
    -- Find employees reporting to each employee
    SELECT
        e.employee_id,
        e.employee_name,
        e.manager_id,
        rc.level + 1,
        CONCAT(
            rc.reporting_path,
            ' -> ',
            e.employee_name
        )
    FROM employee e
    INNER JOIN reporting_chain rc
        ON e.manager_id = rc.employee_id
)
SELECT
    employee_id,
    employee_name,
    manager_id,
    level,
    reporting_path
FROM reporting_chain
ORDER BY reporting_path;
```

OUTPUT:

1: DEPARTMENT SALARY:



```
(102, 'Neha', 1, 101, 65000),
(103, 'Rahul', 1, 101, 60000),
(104, 'Priya', 2, NULL, 80000),
(105, 'Karan', 2, 104, 55000),
(106, 'Riya', 3, NULL, 85000),
(107, 'Arjun', 3, 106, 50000);

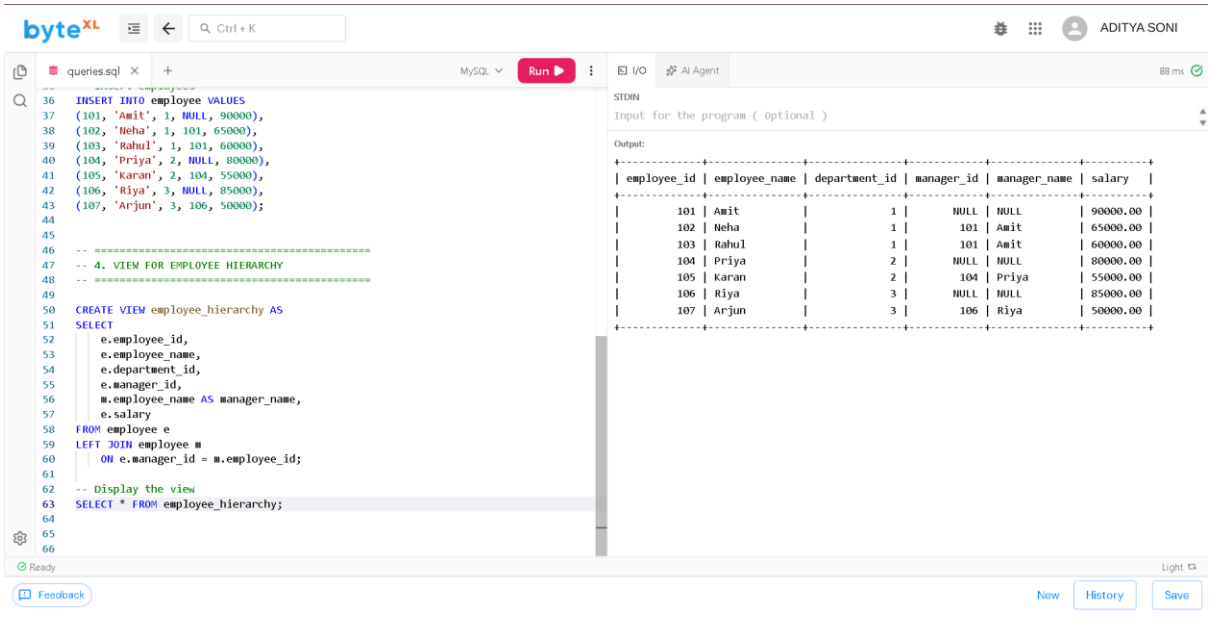

-- ============================================
-- 3. VIEW FOR DEPARTMENT SALARY SUMMARY
-- ============================================

CREATE VIEW department_salary_summary AS
SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS employee_count,
    SUM(e.salary) AS total_salary,
    AVG(e.salary) AS average_salary
FROM department d
LEFT JOIN employee e
    ON d.department_id = e.department_id
GROUP BY
    d.department_id,
    d.department_name;

-- Display the view
SELECT * FROM department_salary_summary;
```

```
+---------------+-----------------+----------------+--------------+----------------+
| department_id | department_name | employee_count | total_salary | average_salary |
+---------------+-----------------+----------------+--------------+----------------+
|             1 | IT              |              3 |    215000.00 |   71666.666667 |
|             2 | HR              |              2 |    135000.00 |   67500.000000 |
|             3 | Finance         |              2 |    135000.00 |   67500.000000 |
+---------------+-----------------+----------------+--------------+----------------+
```

2: EMPLOYEE HEIRARCHY:



```
INSERT INTO employee VALUES
(101, 'Amit', 1, NULL, 90000),
(102, 'Neha', 1, 101, 65000),
(103, 'Rahul', 1, 101, 60000),
(104, 'Priya', 2, NULL, 80000),
(105, 'Karan', 2, 104, 55000),
(106, 'Riya', 3, NULL, 85000),
(107, 'Arjun', 3, 106, 50000);

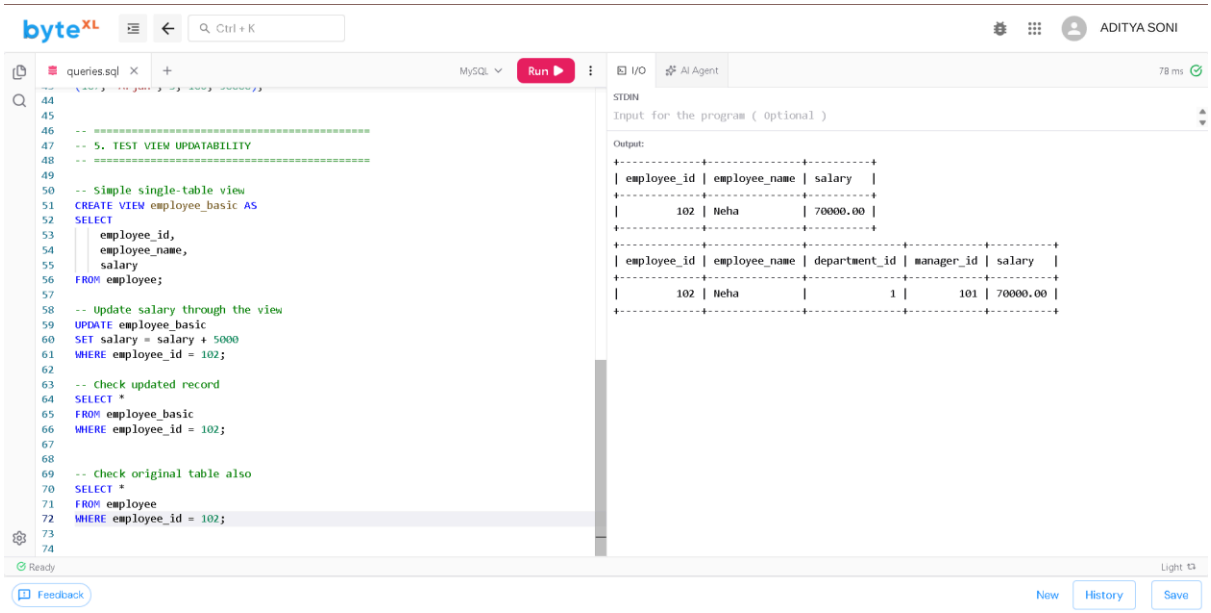

-- ============================================
-- 4. VIEW FOR EMPLOYEE HIERARCHY
-- ============================================

CREATE VIEW employee_hierarchy AS
SELECT
    e.employee_id,
    e.employee_name,
    e.department_id,
    e.manager_id,
    m.employee_name AS manager_name,
    e.salary
FROM employee e
LEFT JOIN employee m
    ON e.manager_id = m.employee_id;

-- Display the view
SELECT * FROM employee_hierarchy;
```

```
+-------------+---------------+---------------+------------+--------------+----------+
| employee_id | employee_name | department_id | manager_id | manager_name | salary   |
+-------------+---------------+---------------+------------+--------------+----------+
|         101 | Amit          |             1 |       NULL | NULL         | 90000.00 |
|         102 | Neha          |             1 |        101 | Amit         | 65000.00 |
|         103 | Rahul         |             1 |        101 | Amit         | 60000.00 |
|         104 | Priya         |             2 |       NULL | NULL         | 80000.00 |
|         105 | Karan         |             2 |        104 | Priya        | 55000.00 |
|         106 | Riya          |             3 |       NULL | NULL         | 85000.00 |
|         107 | Arjun         |             3 |        106 | Riya         | 50000.00 |
+-------------+---------------+---------------+------------+--------------+----------+
```

3: UPDATABILITY TEST:



```sql
-- ============================================
-- 5. TEST VIEW UPDATABILITY
-- ============================================

-- Simple single-table view
CREATE VIEW employee_basic AS
SELECT
    employee_id,
    employee_name,
    salary
FROM employee;

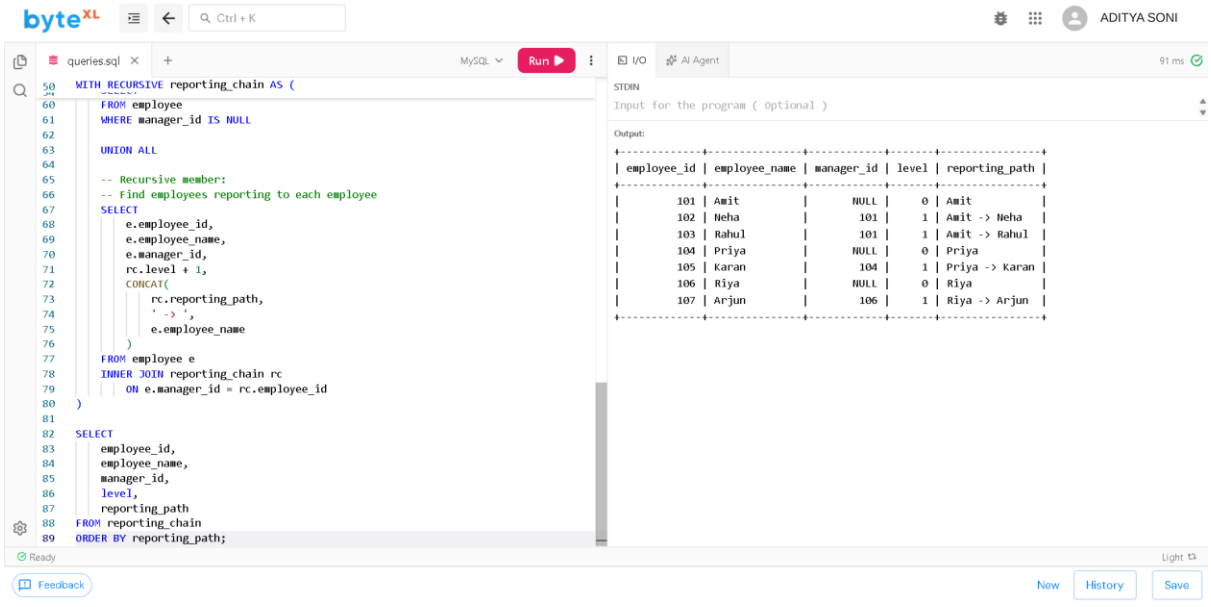
-- Update salary through the view
UPDATE employee_basic
SET salary = salary + 5000
WHERE employee_id = 102;

-- Check updated record
SELECT *
FROM employee_basic
WHERE employee_id = 102;


-- Check original table also
SELECT *
FROM employee
WHERE employee_id = 102;
```

```
+-------------+---------------+----------+
| employee_id | employee_name | salary   |
+-------------+---------------+----------+
|         102 | Neha          | 70000.00 |
+-------------+---------------+----------+
+-------------+---------------+---------------+------------+----------+
| employee_id | employee_name | department_id | manager_id | salary   |
+-------------+---------------+---------------+------------+----------+
|         102 | Neha          |             1 |        101 | 70000.00 |
+-------------+---------------+---------------+------------+----------+
```

4: RECURSIVE CTE REPORTING CHAIN:



```sql
WITH RECURSIVE reporting_chain AS (
    FROM employee
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive member:
    -- Find employees reporting to each employee
    SELECT
        e.employee_id,
        e.employee_name,
        e.manager_id,
        rc.level + 1,
        CONCAT(
            rc.reporting_path,
            ' -> ',
            e.employee_name
        )
    FROM employee e
    INNER JOIN reporting_chain rc
        ON e.manager_id = rc.employee_id
)

SELECT
    employee_id,
    employee_name,
    manager_id,
    level,
    reporting_path
FROM reporting_chain
ORDER BY reporting_path;
```

```
+-------------+---------------+------------+-------+----------------+
| employee_id | employee_name | manager_id | level | reporting_path |
+-------------+---------------+------------+-------+----------------+
|         101 | Amit          |       NULL |     0 | Amit           |
|         102 | Neha          |        101 |     1 | Amit -> Neha   |
|         103 | Rahul         |        101 |     1 | Amit -> Rahul  |
|         104 | Priya         |       NULL |     0 | Priya          |
|         105 | Karan         |        104 |     1 | Priya -> Karan |
|         106 | Riya          |       NULL |     0 | Riya           |
|         107 | Arjun         |        106 |     1 | Riya -> Arjun  |
+-------------+---------------+------------+-------+----------------+
```

Experiment 6

Aim: To create a stored procedure transfer_employee(emp_id, new_dept_id) with validation and error handling, and to implement database triggers for salary validation and audit logging.

Source Code:

```
-- ============================================
-- 1. CREATE DATABASE TABLES
-- ============================================
CREATE TABLE department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL
);
CREATE TABLE employee (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(100) NOT NULL,
    department_id INT,
    salary DECIMAL(10,2),
    FOREIGN KEY (department_id) REFERENCES department(department_id)
);
-- Sample departments
INSERT INTO department VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance'),
(4, 'Marketing');
-- Sample employees
INSERT INTO employee VALUES
(101, 'Rahul', 1, 50000),
(102, 'Aman', 2, 45000),
(103, 'Neha', 1, 60000),
(104, 'Priya', 3, 55000),
(105, 'Rohan', 4, 40000);
-- ============================================
-- 2. CREATE AUDIT TABLE
```

```
-- ============================================
CREATE TABLE employee_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    old_department_id INT,
    new_department_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    action_type VARCHAR(50),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- ============================================
-- 3. STORED PROCEDURE
-- transfer_employee(emp_id, new_dept_id)
-- ============================================

DELIMITER //

CREATE PROCEDURE transfer_employee(
    IN p_emp_id INT,
    IN p_new_dept_id INT
)
BEGIN
    DECLARE v_old_dept_id INT;
    DECLARE v_emp_count INT DEFAULT 0;
    DECLARE v_dept_count INT DEFAULT 0;
    -- Error handling
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error: Transfer failed.' AS message;
    END;
    START TRANSACTION;
    -- Check employee exists
```

```
SELECT COUNT(*)
INTO v_emp_count
FROM employee
WHERE employee_id = p_emp_id;
IF v_emp_count = 0 THEN
   SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Error: Employee does not exist.';
END IF;
-- Check department exists
SELECT COUNT(*)
INTO v_dept_count
FROM department
WHERE department_id = p_new_dept_id;
IF v_dept_count = 0 THEN
   SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Error: New department does not exist.';
END IF;
-- Get current department
SELECT department_id
INTO v_old_dept_id
FROM employee
WHERE employee_id = p_emp_id;
-- Check if already in department
IF v_old_dept_id = p_new_dept_id THEN
   SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Error: Employee is already in this department.';
END IF;
-- Transfer employee
UPDATE employee
SET department_id = p_new_dept_id
WHERE employee_id = p_emp_id;
COMMIT;
SELECT 'Employee transferred successfully.' AS message;
```

```
END //
DELIMITER ;
-- ============================================
-- 4. TRIGGER FOR SALARY VALIDATION
-- ============================================
DELIMITER //
CREATE TRIGGER before_employee_insert
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Salary cannot be negative.';
    END IF;
END //
DELIMITER ;
-- ============================================
-- 5. TRIGGER FOR SALARY UPDATE VALIDATION
-- ============================================
DELIMITER //
CREATE TRIGGER before_employee_salary_update
BEFORE UPDATE ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Salary cannot be negative.';
    END IF;
END //
DELIMITER ;
-- ============================================
-- 6. AUDIT TRIGGER
-- Logs department transfers
-- ============================================
```

```
DELIMITER //
CREATE TRIGGER after_employee_update
AFTER UPDATE ON employee
FOR EACH ROW
BEGIN
   IF OLD.department_id <> NEW.department_id THEN
      INSERT INTO employee_audit
      (
         employee_id,
         old_department_id,
         new_department_id,
         old_salary,
         new_salary,
         action_type
      )
      VALUES
      (
         OLD.employee_id,
         OLD.department_id,
         NEW.department_id,
         OLD.salary,
         NEW.salary,
         'DEPARTMENT TRANSFER'
      );
   END IF;
END //
DELIMITER ;
-- ============================================
-- 7. TESTING
-- ============================================
-- Test 1: Valid transfer
CALL transfer_employee(101, 3);
-- Check employee
SELECT * FROM employee WHERE employee_id = 101;
```

-- Check audit

SELECT * FROM employee_audit;

-- Test 2: Invalid employee

CALL transfer_employee(999, 2);

-- Test 3: Invalid department

CALL transfer_employee(102, 999);

-- Test 4: Employee already belongs to department

CALL transfer_employee(103, 1);

-- Test 5: Negative salary during INSERT

INSERT INTO employee

VALUES (106, 'Test Employee', 1, -5000);

-- Test 6: Negative salary during UPDATE

UPDATE employee

SET salary = -1000

WHERE employee_id = 102;

-- ==========

-- 8. FINAL DATA

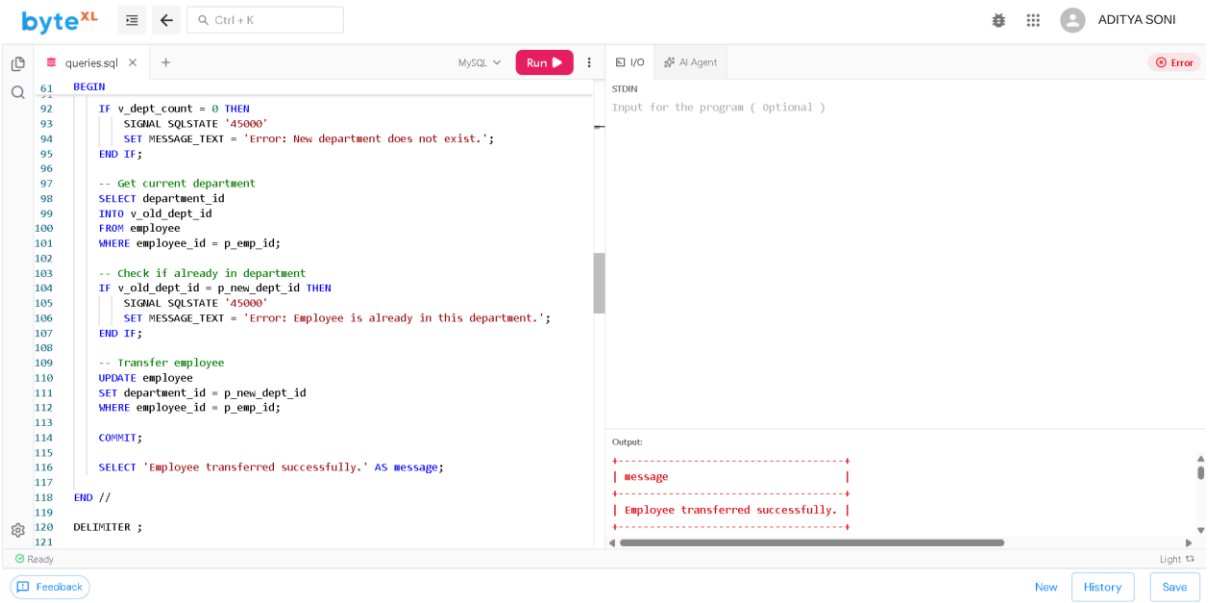
-- ==========

SELECT * FROM employee;

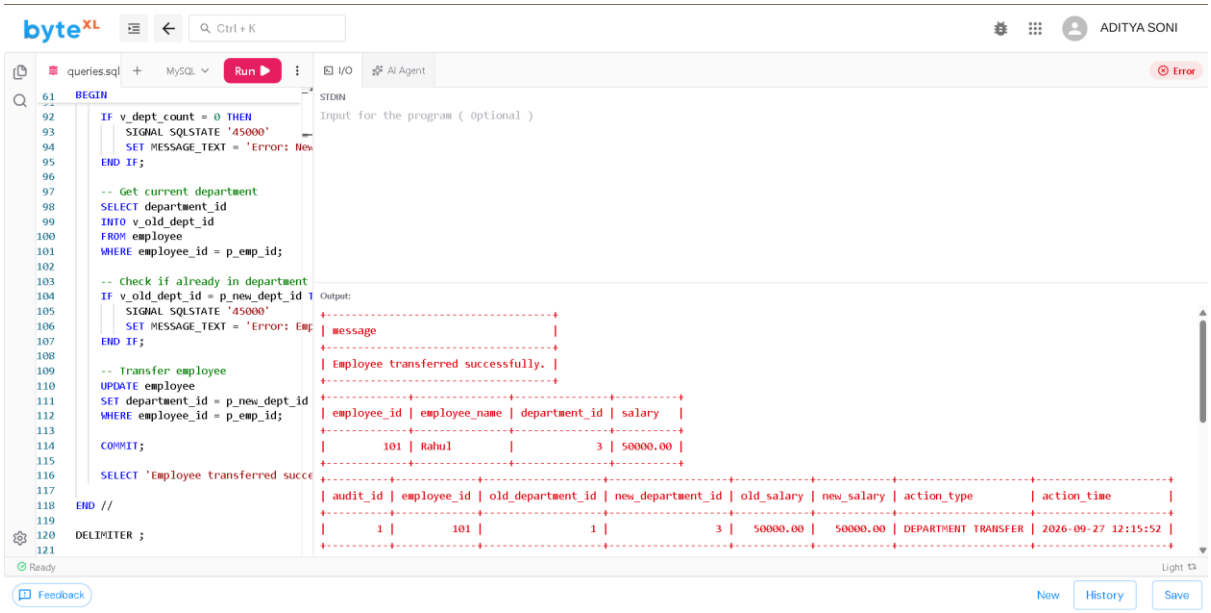
SELECT * FROM employee_audit;

OUTPUT:

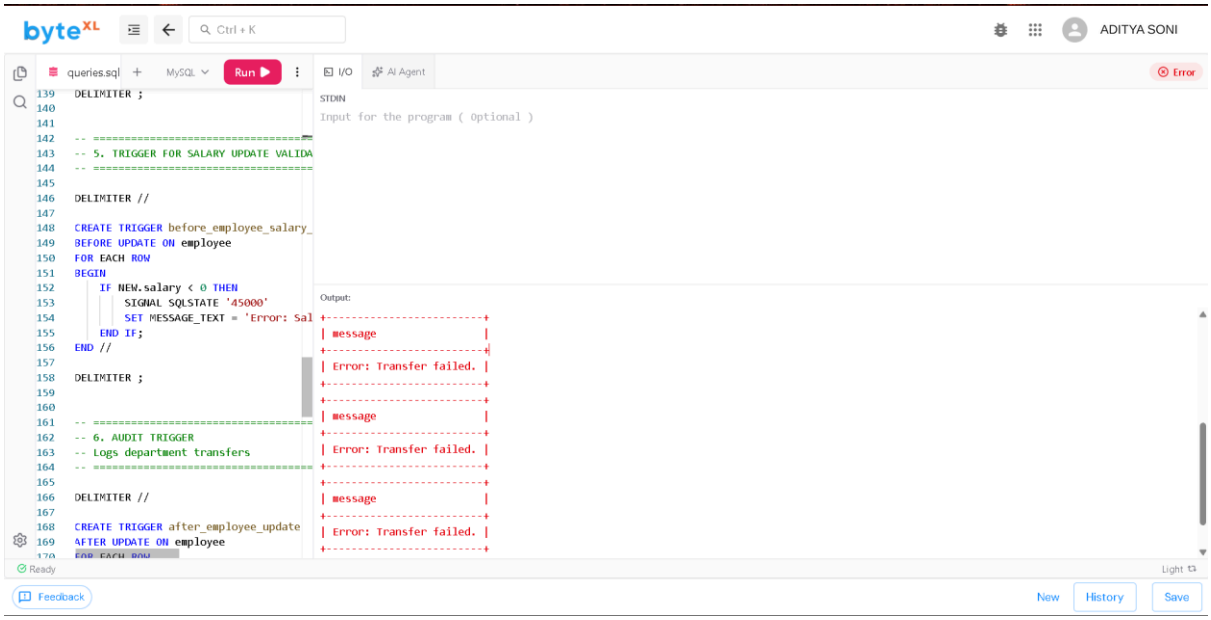
1: SUCCESSFUL EMPLOYEE TRANSFER:



2: INVALID EMPLOYEE / DEPARTMENT VALIDATION:



3: SALARY VALIDATION TRIGGER:



4: SALARY AUDIT LOG:

